WEEKLY PLAN • 2026 W46

本周工作安排

后端开发（初级）· 会员 & 积分域 · Java / Spring Boot

Leader 派发 · 周一 09:30 生效

周期	2026-11-09（周一）— 2026-11-13（周五），共 5 个工作日
主线需求	会员积分明细查询接口 v2（多条件筛选 + 分页 + 缓存）
需求单号	PRD-20431 / 技术任务 AONE-88217（子任务 5 个，见第二章清单）
代码分支	`feature/point_detail_v2`（由 develop 切出，禁止直推主干）
交付环境	日常 → 预发（pre）→ 生产灰度 → 全量
本周产出	1 个上线接口、1 份设计文档、1 份压测报告、1 份复盘

本周一句话目标

周五 18:00 前，把「积分明细查询 v2」安全推上生产并灰度到 100%：接口 P99 ≤ 120ms（200 QPS 压测），单测行覆盖率 ≥ 70%，线上零回滚、零告警。

DELIVERY

交付闭环

需求 → 设计 → 编码 → 测 → 灰度，每一环都有可验收的产出物，不留"我以为做完了"。

QUALITY

质量内建

单测、越权校验、缓存三防写在代码里，而不是等 QA 或线上事故来提醒你。

VISIBILITY

可观测

每上一个功能，都配一条 RT 曲线和一个告警阈值——出事时你有 30 秒定位的能力。

关于这份排期：每天的任务不是"写多少行代码"，而是"今天结束时能拿出什么给别人看"。所以每一天都给了**目标、任务、具体实现（代码/SQL/命令）、交付物、验收口径、易踩的坑**。做不完可以砍范围，但不要砍验收——砍掉验收等于把风险推到周五晚上。

01 本周总纲

先讲清楚"为什么这么排"，再讲"每天做什么"。周一早上花 10 分钟读完这一章，比后面五天盲目写代码有用。

1.1 三条线并行，精力按 60 / 25 / 15 分配

线路	内容	占比	为什么给你这条线
主线 必须交付	积分明细查询接口 v2 的完整交付	60%	一个"麻雀虽小五脏俱全"的需求：有读有写、有分页、有缓存、有权限、要上线。练的是完整闭环，不是纯写 CRUD。
辅线 顺手做	积分域存量接口的稳定性治理：慢 SQL、无缓存热点接口、缺失的监控	25%	初级开发最容易只盯自己那一亩地。让你看一眼老接口，才知道"能跑"和"跑得好"差在哪。
成长线 每天 30min	COLA 分层规范、缓存三防、灰度与回滚三板斧	15%	这三样是团队里"能不能独立负责一个模块"的门槛。不要求精通，要求能讲清、能照着做。

1.2 关键结果（KR）——周五拿什么证明你做完了

编号	关键结果	验收口径（可量化，别写"基本完成"）
KR1	接口按时上线	周五 17:00 前生产灰度 100%，变更单闭合，无遗留 TODO 上线
KR2	性能达标	预发压测 200 QPS 持续 10 分钟：P99 ≤ 120ms、错误率 < 0.01%、CPU < 60%
KR3	单测覆盖	新增代码行覆盖率 ≥ 70%，核心分支（分页边界、空结果、越权、缓存击穿）≥ 85%
KR4	SQL 健康	新增 SQL 全部走索引，执行计划无 `filesort` / `Using temporary`，无 > 100ms 慢 SQL
KR5	Code Review	CR 意见 ≤ 5 条，且无"严重/阻断"级问题；一次通过率 100%
KR6	文档齐全	设计文档、接口文档、压测报告、上线复盘四份齐全，链接挂在任务单上

1.3 每天的固定节奏（比任务清单更重要）

时间	动作	要求
09:30 - 09:40	站会	三句话：昨天完成了什么、今天做什么、卡在哪。**卡点必须当场说**，不要憋到下午。
09:40 - 12:00	深度编码	关掉钉钉免打扰，只做当天最难的那件事。上午不处理零碎消息。
13:30 - 17:30	联调 / 自测 / 改 CR	下午留给需要别人配合的事。找人之前先自己复现一遍。
17:30 - 18:00	提交代码	当天代码必须提交到 feature 分支，commit message 写清"做了什么 + 为什么"。
18:00 - 18:30	成长线 30 分钟	看一篇团队文档 / 读一段老代码 / 把今天踩的坑记到笔记里。
周五 16:00	复盘	30 分钟，写"哪件事返工了、根因是什么、下次怎么避免"。

1.4 协作地图：什么时候该找谁

角色	找他干什么	最佳时机	怎么问（示例）
我（Leader）	需求边界不清、方案取舍、要资源	周一 10:30 前 / 随时阻塞时	"这块有两个方案 A/B，我倾向 A，因为……你帮我确认下方向对不对。"

产品（PD）	字段含义、筛选条件口径、异常文案	周一澄清会一次性问完	带上具体数据截图问，别问"这个字段啥意思"。
QA	用例评审、预发验证、灰度观察	周三下午（编码完成前）	"我周三提测，用例里这 3 个边界你帮我重点看下。"
DBA / 运维	DDL 评审、索引评估、变更发布	周二建表前	附上 DDL + 预估数据量 + 查询 SQL。
师兄 / 同组老同学	不懂的框架用法、历史坑	自己查 30 分钟无果后	"我查了文档是这样，但实测不生效，可能是我漏了哪一步？"

1.5 风险与预案（提前想好，别周五才发现）

风险	概率	影响	预案
数据量比预估大，分页性能塌	中	高	周二先用生产脱敏数据量做 `explain`；不行就上"游标分页 + 只允许翻前 200 页"，周三前定稿。
需求中途加筛选条件	高	中	周三 12:00 后需求冻结，新增一律进 v2.1。变更走我确认，不私下接。
缓存与 DB 不一致	中	中	只缓存"明细类只读数据"，TTL 5 分钟兜底；写路径不做主动删缓存，靠 TTL 收敛。
灰度期间 RT 上涨	低	高	Diamond 开关可秒级关闭缓存失败降级；准备一键回滚 SOP（见周五）。
自己估时过于乐观	高	中	每项任务按"我的估计 × 1.5"排；每天 17:30 对一次进度，落后立刻砍范围。

02 本周任务清单

共 12 项，编号即任务单子任务编号。P0 = 本周必须完成，P1 = 应该完成，P2 = 有余力再做。

编号	任务	类型	优先	预估	交付物 / 完成标志
T1	需求澄清与口径对齐	需求	**P0**	1h	澄清纪要（8 个问题的答案）发群里，PD 确认
T2	技术方案设计（模型 / 接口 / 表 / 缓存）	设计	**P0**	3h	设计文档 v1，含时序图与索引设计，我 review 通过
T3	建表 DDL + 索引 + DBA 评审	DB	**P0**	1.5h	DDL 脚本 + DBA 评审意见已回
T4	工程骨架与分层搭建（COLA）	编码	**P0**	1.5h	分支可编译，5 个模块目录建好
T5	查询主链路：Controller → Exe → Gateway → Mapper	编码	**P0**	6h	日常环境接口可通，Postman 能返回正确数据
T6	越权校验与数据脱敏	安全	**P0**	1.5h	换 userId 调接口返回 403；脱敏字段已掩码
T7	缓存层（穿透 / 击穿 / 雪崩三防）	编码	**P0**	3h	命中率 ≥ 60%，空结果也缓存，TTL 带随机抖动
T8	单元测试 + 覆盖率报告	质量	**P0**	4h	覆盖率 ≥ 70%，Jacoco 报告链接
T9	提测 + 联调 + 缺陷修复	测试	**P0**	5h	QA 用例全通过，P0/P1 缺陷清零
T10	压测 + 慢 SQL 治理 + CR 修改	质量	**P0**	5h	压测报告（P99/错误率/CPU）；CR 意见全部关闭
T11	灰度发布 + 监控看板 + 告警	上线	**P0**	3h	灰度 1%→10%→100%，看板与 4 条告警生效
T12	辅线：存量积分查询接口治理	治理	**P1**	4h	治理清单（≥ 3 条问题 + 修复建议），能改的先改
T13	周五复盘 + 下周计划	管理	**P1**	1h	复盘文档 1 页，下周 3 个候选任务

读这张表的正确方式：P0 全部是"串联"的——T2 没定稿，T5 写了也要返工；T8 没做，T9 提测就是让 QA 帮你做单测。所以顺序比速度重要。**预估工时已经乘过 1.5 的系数**，如果实际超了 30%，当天 17:30 就来跟我说，不要自己扛。

03 每日任务与具体实现

每一天：目标 → 任务 → 具体实现（可直接抄的代码 / SQL / 命令）→ 交付物 → 验收 → 坑。代码只是示意结构，包名与内部框架以团队现有工程为准。

DAY 01 周一：把"要做什么"变成"怎么写"　11-09 / 设计日

今日目标：下班前拿出一份能过评审的设计文档。今天一行业务代码都不用写，写错了才最贵。

任务

- **T1 需求澄清（1h）**：把下面 8 个问题问清楚，答案写进纪要发群里让 PD 回复确认。
- **T2 方案设计（3h）**：领域模型、接口契约、表结构与索引、缓存策略、异常码。
- **T3 建表评审（1.5h）**：DDL 提给 DBA，重点问索引是否够、分表策略要不要现在做。
- **成长线（30min）**：读一遍团队 COLA 分层规范，能说出 adapter / app / domain / infrastructure 各自不该出现什么代码。

具体实现

① 需求澄清必须问的 8 个问题

- 筛选条件之间是 AND 还是 OR？时间范围默认值是什么，最大跨度允许多少天？
- 分页是必须支持任意翻页，还是只需要"加载更多"？最大页码有没有限制？
- 积分明细的数据来源是单表还是多表（明细表 + 汇总表）？以哪张为准？
- 历史数据怎么处理：老数据保留在原表，还是需要迁移/合并展示？
- 查询为空时返回空列表还是报错？前端需要区分吗？
- 哪些字段属于敏感信息（手机号、身份证、余额）？需要脱敏到什么粒度？
- 接口 QPS 预期多少？有没有大促/活动带来的尖峰？
- 上线时间点有没有硬约束（是否要避开交易高峰）？

② 表结构与索引（T3 交付）

```
-- 积分明细表：按用户维度查询为主，故以 user_id 为索引首列
CREATE TABLE `point_detail` (
  `id`           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '主键',
  `user_id`      BIGINT UNSIGNED NOT NULL                COMMENT '用户ID',
  `biz_type`     TINYINT UNSIGNED NOT NULL DEFAULT 0     COMMENT '业务类型 1消费 2签到 3退款 4活动',
  `change_amount` INT NOT NULL                           COMMENT '变动积分，正加负减',
  `balance`      INT NOT NULL                            COMMENT '变动后余额',
  `biz_no`       VARCHAR(64) NOT NULL DEFAULT ''         COMMENT '业务单号，幂等键',
  `remark`       VARCHAR(128) NOT NULL DEFAULT ''        COMMENT '备注',
  `gmt_create`   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `gmt_modified` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_biz` (`biz_no`, `biz_type`),            -- 幂等
  KEY `idx_user_time` (`user_id`, `gmt_create`)          -- 覆盖主查询：user + 时间倒序
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='积分明细表';
```

易踩的坑：① `idx_user_time` 顺序不能反，反了就用不上排序，会多一次 filesort。② 别把 `biz_type` 塞到索引第二列——它的选择性太低，加了反而让优化器选错索引。③ 建表语句必须先给 DBA 看再执行，生产 DDL 走变更平台，不要手连数据库。

③ 接口契约（先定契约，再写实现）

```
// client 模块，对外暴露的 API
public interface PointDetailService {
    Result> pageQuery(PointDetailQuery query);
}

@Data
public class PointDetailQuery implements Serializable {
    private Integer pageNo = 1;          // 从 1 开始
    private Integer pageSize = 20;       // 上限 50，服务端强校验
    private Integer bizType;             // 可空，空=全部
    private Date startTime;              // 可空，默认近 90 天
    private Date endTime;
    // 注意：绝对不要暴露 userId 字段，用户身份从登录态取
}

@Data
public class PointDetailDTO {
    private Long id;
    private Integer bizType;
    private String bizTypeDesc;
    private Integer changeAmount;
    private Integer balance;
    private String remark;
    private String gmtCreate;            // 统一格式化为 yyyy-MM-dd HH:mm:ss
}
```

④ 缓存 Key 设计（周一就定，周三直接实现）

Key 模板	TTL	说明
`point:detail:{userId}:{bizType}:{start}:{end}:{pageNo}:{pageSize}`	5min	只缓存 pageNo ≤ 3 的请求；TTL 加 ±10% 随机抖动
`point:detail:empty:{userId}:{hash}`	60s	空结果缓存，防穿透（用短 TTL，避免数据补录后长期看不到）

交付物与验收

- **交付物：**澄清纪要（群消息）、设计文档 v1（含 DDL、接口定义、缓存设计、异常码表）、DBA 评审记录。
- **验收：**我能只读文档就说出这个接口的输入输出和边界；DBA 在 DDL 上签了"可执行"。

DAY 02 周二：搭骨架，把主链路打通

11-10 / 编码日

今日目标：日常环境能跑通一次真实查询，返回结构和文档一致。先跑通，再优化——今天不要碰缓存。

任务

- **T4 工程骨架（1.5h）**：按 COLA 分层建包，配置好 boot 启动类与数据源。
- **T5 主链路（6h）**：Controller / Exe / Gateway / Mapper 四层写完，参数校验 + 分页 + 排序。
- **T6 越权校验（1.5h）**：身份从登录态取，入参出现 userId 一律忽略并打日志。
- **成长线（30min）**：搞清 `@Resource` 与 `@Autowired` 在你们工程里的使用约定，别两种混用。

具体实现

① 分层结构（COLA 4.x）

```
point-service
├── point-client          // 对外二方包：DTO / Query / Service 接口 / 异常码
├── point-adapter         // 接入层：Controller、VO、参数校验、异常转换。禁止写业务逻辑
├── point-app             // 应用层：Exe 编排（查缓存→查库→组装），事务边界在这层
├── point-domain          // 领域层：实体、值对象、Gateway 接口。禁止出现 MyBatis / RPC 注解
└── point-infrastructure  // 基础设施：Mapper、GatewayImpl、缓存、外部服务调用
```

评审必看的一条：domain 层如果 import 了 `org.apache.ibatis` 或 `com.taobao.hsf`，直接算不合格。分层的意义是"换存储/换 RPC 不动业务"，混了就白分。

② Controller：只做三件事——接参、校验、转换

```
@RestController
@RequestMapping("/api/point")
public class PointDetailController {

    @Resource private PointDetailPageExe pointDetailPageExe;

    @GetMapping("/detail/page")
    public Result> page(PointDetailQuery query) {
        // 1. 身份只从登录态取，不信任前端传入
        Long userId = LoginContext.getUserId();
        Assert.notNull(userId, "未登录");

        // 2. 参数纠偏（在边界做，别把脏数据带到下游）
        query.setPageNo(query.getPageNo() == null || query.getPageNo() < 1 ? 1 : query.getPageNo());
        query.setPageSize(query.getPageSize() == null ? 20 : Math.min(query.getPageSize(), 50));

        // 3. 交给应用层，Controller 里不出现任何 if-业务判断
        PageResult<PointDetailDTO> page = pointDetailPageExe.execute(userId, query);
        return Result.success(PageResult.of(page, PointDetailVO::from));
    }
}
```

③ 应用层 Exe：今天先写"无缓存版"，明天再加缓存

```
@Component
public class PointDetailPageExe {

    @Resource private PointDetailGateway pointDetailGateway;

    public PageResult<PointDetailDTO> execute(Long userId, PointDetailQuery query) {
        // 1. 时间窗兜底：不传默认近 90 天，跨度超 1 年直接拒绝（防拖库）
        DateRange range = DateRange.ofDefault(query.getStartTime(), query.getEndTime(), 90);
        Assert.isTrue(range.days() <= 365, "查询时间跨度不能超过一年");

        // 2. 先查 count，再判断是否越界（避免无意义的深分页查询）
        long total = pointDetailGateway.countByUser(userId, query.getBizType(), range);
        if (total == 0) {
            return PageResult.empty(query.getPageNo(), query.getPageSize());
        }
        int maxPage = (int) Math.ceil(total * 1.0 / query.getPageSize());
        Assert.isTrue(query.getPageNo() <= maxPage, "页码超出范围");

        // 3. 查列表
        List<PointDetailDTO> list = pointDetailGateway.pageByUser(
                userId, query.getBizType(), range, query.getPageNo(), query.getPageSize());
        return PageResult.of(list, total, query.getPageNo(), query.getPageSize());
    }
}
```

④ Mapper：分页 + 索引友好写法

```xml
<select id="pageByUser" resultMap="detailMap">
  SELECT id, biz_type, change_amount, balance, remark, gmt_create
  FROM point_detail
  WHERE user_id = #{userId}
    AND gmt_create BETWEEN #{start} AND #{end}
  <if test="bizType != null"> AND biz_type = #{bizType} </if>
  ORDER BY gmt_create DESC, id DESC      <!-- 加 id 兜底，防同秒乱序 -->
  LIMIT #{offset}, #{pageSize}
</select>

<!-- count 单独写，别用 select count(1) 套整段 SQL -->
<select id="countByUser" resultType="long">
  SELECT COUNT(1) FROM point_detail
  WHERE user_id = #{userId} AND gmt_create BETWEEN #{start} AND #{end}
  <if test="bizType != null"> AND biz_type = #{bizType} </if>
</select>
```

深分页警报：当 `offset` 上万时，MySQL 会扫描并丢弃前面的行，RT 直接飙到几百毫秒。今天先跑通，如果 explain 显示扫描行数超过 5000，**周三前**改成游标分页：`WHERE user_id=? AND (gmt_create,id) < (#{lastTime},#{lastId}) ORDER BY gmt_create DESC,id DESC LIMIT ?`，前端改成"加载更多"。

⑤ 自测命令

```bash
# 本地起服务后自测（把 token 换成自己的）
curl -s 'http://localhost:8080/api/point/detail/page?pageNo=1&pageSize=20&bizType=1' \
  -H 'Cookie: login_token=xxx' | python -m json.tool

# 直连日常库验证 SQL 走不走索引
EXPLAIN SELECT id, biz_type, change_amount, balance, remark, gmt_create
FROM point_detail WHERE user_id = 10086
  AND gmt_create BETWEEN '2026-08-01' AND '2026-11-09'
ORDER BY gmt_create DESC, id DESC LIMIT 0, 20;
# 期望：type=range, key=idx_user_time, Extra 里没有 Using filesort / Using temporary
```

交付物与验收

- **交付物：**feature 分支可编译可启动；Postman/curl 能返回正确 JSON；explain 截图一张。
- **验收：**pageNo 传 0、pageSize 传 1000、时间跨度 3 年、未登录四种情况，服务端都不报 500，而是返回明确提示。

DAY 03 周三：缓存与单测，把质量焊进代码

11-11 / 质量日

今日目标：接口有缓存、有单测、覆盖率达标。今天下午 15:00 提测给 QA，晚一分钟就是明天的债。

任务

- **T7 缓存三防（3h）**：穿透用空值缓存、击穿用单飞锁、雪崩用 TTL 抖动。
- **T8 单元测试（4h）**：覆盖分页边界、空结果、越权、缓存异常降级四类场景。
- **提测（0.5h）**：写好提测说明，附改动点、影响面、自测结论、重点回归项。
- **成长线（30min）**：把"缓存三防"用自己的话写成三句话，明天能讲给 QA 听。

具体实现

① 缓存层：三防一次写全

```
@Component
public class PointDetailCache {

    @Resource private TairClient tair;          // 团队一般用 Tair / Redis 封装
    private static final int TTL_SECONDS = 300;

    public PageResult<PointDetailDTO> get(String key) {
        try {
            return tair.get(key);
        } catch (Exception e) {
            // 缓存故障不能拖垮查询：记日志 + 降级走 DB
            log.warn("point_cache_get_fail, key={}", key, e);
            return null;
        }
    }

    public void put(String key, PageResult<PointDetailDTO> value) {
        // 防雪崩：TTL 加 ±10% 随机抖动，避免同一秒集体失效
        int ttl = TTL_SECONDS + ThreadLocalRandom.current().nextInt(30) - 15;
        try {
            tair.setex(key, ttl, value);
        } catch (Exception e) {
            log.warn("point_cache_put_fail, key={}", key, e);
        }
    }

    public void putEmpty(String key) {
        try {
            tair.setex(key, 60, PageResult.empty(1, 20));   // 防穿透：短 TTL
        } catch (Exception ignore) { }
    }
}
```

② 在 Exe 里接入缓存（注意只缓存前 3 页）

```
public PageResult<PointDetailDTO> execute(Long userId, PointDetailQuery query) {
    DateRange range = DateRange.ofDefault(query.getStartTime(), query.getEndTime(), 90);
    Assert.isTrue(range.days() <= 365, "查询时间跨度不能超过一年");

    // 深分页不缓存：key 爆炸且命中率极低
    boolean cacheable = query.getPageNo() <= 3;
    String key = PointCacheKey.build(userId, query, range);

    if (cacheable) {
        PageResult<PointDetailDTO> cached = pointDetailCache.get(key);
        if (cached != null) {
            return cached;
        }
    }

    long total = pointDetailGateway.countByUser(userId, query.getBizType(), range);
    if (total == 0) {
        if (cacheable) pointDetailCache.putEmpty(key);
        return PageResult.empty(query.getPageNo(), query.getPageSize());
    }
    // ... 查列表、组装
    PageResult<PointDetailDTO> result = PageResult.of(list, total, query.getPageNo(), query.getPageSize());
    if (cacheable) pointDetailCache.put(key, result);
    return result;
}
```

缓存一致性怎么答（会被问）： 本接口只读历史明细，写入方是积分变动服务，允许 5 分钟内的延迟（已与 PD 确认）。因此不做主动删缓存，靠 TTL 收敛。如果以后要求强一致，改成"写后删 key + 延迟双删"，那时再讨论。

③ 单元测试：先写"最可能出错"的四类

```
@ExtendWith(MockitoExtension.class)
class PointDetailPageExeTest {

    @Mock private PointDetailGateway pointDetailGateway;
    @Mock private PointDetailCache pointDetailCache;
    @InjectMocks private PointDetailPageExe exe;

    @Test
    @DisplayName("空结果：返回空列表且写入空值缓存")
    void shouldReturnEmptyAndCacheNull() {
        when(pointDetailGateway.countByUser(anyLong(), any(), any())).thenReturn(0L);

        PageResult<PointDetailDTO> r = exe.execute(10086L, defaultQuery());

        assertTrue(r.getList().isEmpty());
        verify(pointDetailCache).putEmpty(anyString());
    }

    @Test
    @DisplayName("时间跨度超一年：直接抛参数异常，不打库")
    void shouldRejectTooLongRange() {
        PointDetailQuery q = defaultQuery();
        q.setStartTime(DateUtil.offsetYear(new Date(), -2));

        assertThrows(IllegalArgumentException.class, () -> exe.execute(10086L, q));
        verifyNoInteractions(pointDetailGateway);       // 关键断言
    }

    @Test
    @DisplayName("缓存命中：不查库")
    void shouldHitCache() {
        when(pointDetailCache.get(anyString())).thenReturn(PageResult.of(List.of(mockDto()), 1, 1, 20));

        exe.execute(10086L, defaultQuery());

        verifyNoInteractions(pointDetailGateway);
    }

    @Test
    @DisplayName("页码越界：抛异常而不是返回空")
    void shouldRejectOutOfRangePage() {
        when(pointDetailGateway.countByUser(anyLong(), any(), any())).thenReturn(5L);
        PointDetailQuery q = defaultQuery();
        q.setPageNo(99);

        assertThrows(IllegalArgumentException.class, () -> exe.execute(10086L, q));
    }
}
```

④ 跑覆盖率

```
mvn clean test jacoco:report -pl point-app -am
# 报告：point-app/target/site/jacoco/index.html
# 只关心新增代码覆盖率，历史代码不用现在补
```

交付物与验收

- **交付物**：提测邮件/消息（改动点 + 影响面 + 自测结论 + 重点回归项）、Jacoco 覆盖率截图。
- **验收**：覆盖率 ≥ 70%；把缓存关掉（配置开关）后接口仍能正常返回；QA 能独立按提测说明复现你的自测步骤。

DAY 04 周四：联调、压测、挨 CR

11-12 / 打磨日

今日目标：预发环境跑出真实性能数据，把 CR 意见全部关闭。今天的产出是"数据"，不是"代码"。

任务

- **T9 联调与缺陷修复（3h）**：跟着 QA 过用例，P0/P1 缺陷当天清零。
- **T10 压测（2h）**：200 QPS × 10min，盯 RT、错误率、CPU、缓存命中率。
- **T10 慢 SQL 治理（1.5h）**：explain 全部新增 SQL，有 filesort 就改索引或改 SQL。
- **CR（1.5h）**：发起 review，按下面的自查表先自己过一遍再提。

具体实现

① 提 CR 前的自查表（能省掉一半的评审意见）

#	检查项	不合格示例
1	没有魔法值	`if (bizType == 1)` → 用枚举 `BizType.CONSUME`
2	没有吞异常	`catch (Exception e) {}` 是红线，至少 log.warn 并给出降级行为
3	日志可排查	打日志带 userId、key、耗时；不打全量对象、不打敏感字段
4	NPE 已防	拆箱、Map.get、远程返回都可能为 null，逐个判
5	方法不过长	单方法超 80 行、嵌套超 3 层，抽方法
6	无调试残留	System.out、TODO、写死的测试数据、注释掉的代码块
7	事务边界正确	查询接口不开事务；开事务的方法里不做 RPC/HTTP 调用
8	兼容性	新增字段是否影响老调用方；二方包改动是否向后兼容

② 压测：先单机，再集群

```
# 单接口压测（200 并发，持续 10 分钟，目标 200 QPS）
jmeter -n -t point_detail.jmx -l result.jtl \
  -Jthreads=200 -Jduration=600 -Jhost=pre-point.example.com

# 或用 wrk（更轻）
wrk -t4 -c100 -d600s --latency \
  -H "Cookie: login_token=xxx" \
  "http://pre-point.example.com/api/point/detail/page?pageNo=1&pageSize=20"
```

观测项	目标	不达标时先看什么
P99 RT	≤ 120ms	先看是 DB 慢还是缓存未命中；用 Arthas `trace` 打方法耗时
错误率	< 0.01%	看异常日志 TOP1；大概率是连接池打满或超时配置太小
缓存命中率	≥ 60%	低于 60% 说明 key 设计有问题（如带时间戳导致永不命中）
CPU / Load	< 60%	CPU 高但 RT 不高：GC 频繁，看 young gc 次数与耗时
DB 慢查询	0 条 > 100ms	explain 看 rows 与 Extra，缺索引就补，深分页就改游标

③ 线上定位三板斧（Arthas）

```
# 1. 看方法耗时分布，定位是哪一层慢
trace com.example.point.app.PointDetailPageExe execute '#cost > 50'

# 2. 看入参出参，复现线上脏数据（只打一次，别刷）
watch com.example.point.app.PointDetailPageExe execute \
  '{params, returnObj}' -x 2 -n 1

# 3. 反编译确认线上跑的到底是不是你提交的代码
sc -d com.example.point.app.PointDetailPageExe | grep code-source
```

交付物与验收

- **交付物：**压测报告（QPS / P99 / 错误率 / CPU / 命中率五张图）、CR 意见关闭记录、explain 截图。

- **验收**：五项指标全部达标；CR 无"严重"级问题；QA 给出"可以上线"的结论。

> **今天最容易翻车的地方**：压测数据用真实用户分布，别全压同一个 userId——单用户数据量小，缓存全命中，压出来的 P99 是假的。至少准备 1 万个 userId 做参数化。

DAY 05 周五：灰度上线、盯盘、复盘

11-13 / 上线日

今日目标：安全上线并灰度到 100%，全程可观测、可回滚。今天你的角色从"开发者"变成"值班人"。

任务

- **T11 变更发布（1.5h）**：按三板斧逐级灰度，每级观察不少于 15 分钟。
- **T11 监控与告警（1h）**：配 4 条告警，自己先手动触发一次验证真的会响。
- **T12 辅线治理（2h）**：翻 3 个存量接口，产出治理清单。
- **T13 复盘（1h）**：16:00 开始，写完发我。

具体实现

① 上线三板斧：可灰度、可监控、可回滚

维度	本周怎么做	验收标志
可灰度	按机器分组发布：1 台 → 10% → 50% → 100%	每级观察 ≥ 15min，指标无异常才进下一级
可监控	接口 RT / QPS / 错误率 / 缓存命中率四条曲线 + 4 条告警	人为触发一次告警（如错误率阈值调低），确认能收到
可回滚	Diamond 开关控制"是否走新逻辑"，无需重新发布	开关关掉后 1 分钟内流量回到老逻辑

② 降级开关（上线前必须先合入）

```
@Component
public class PointDetailSwitch {

    // Diamond 配置：point.detail.v2.enable，默认 true
    @DiamondValue(key = "point.detail.v2.enable", defaultValue = "true")
    private boolean enableV2;

    public boolean useV2() { return enableV2; }
}

// 在 Controller 入口分流：关掉即回老接口，零发布回滚
if (pointDetailSwitch.useV2()) {
    return pointDetailPageExe.execute(userId, query);
}
return legacyService.page(userId, query);
```

③ 告警配置（阈值先保守，跑一周再收紧）

指标	阈值	级别	处理动作
接口错误率	> 0.5% 持续 3min	P1	先看异常日志，确认是否新代码引起 → 关开关
P99 RT	> 300ms 持续 5min	P2	看 DB 慢查询与缓存命中率
缓存命中率	< 40% 持续 10min	P2	查 key 是否异常（如时间戳精度导致不命中）

慢 SQL 数	> 1 条/分钟	P1	拉 explain，必要时限流

④ 灰度节奏（写进变更单，按表执行）

```
10:00  变更单评审通过，确认回滚方案与值班人
10:30  灰度 1 台（约 1%）：观察 20min，看错误率与 RT
11:00  灰度 10%：观察 30min
14:00  灰度 50%：观察 30min，同步 QA 回归核心用例
15:00  全量 100%：观察 1h
16:00  变更单闭合，写复盘

# 任意一级出现以下任一情况，立即停止并回滚：
# 错误率 > 0.5% / P99 > 300ms / 出现新的异常类型 / QA 反馈功能异常
```

⑤ 回滚 SOP（贴在变更单里，别只记在脑子里）

- 第一步：Diamond 关闭 `point.detail.v2.enable`，观察 2 分钟——90% 的问题到这一步就止血了。
- 第二步：若仍异常，回滚应用版本（发布平台一键回滚到上一版本），观察 5 分钟。
- 第三步：若涉及 DDL（本次只新增表，无破坏性变更），确认无写入异常后保留表，不删。
- 第四步：在群里同步"已回滚 + 现象 + 初步原因"，不猜、不隐瞒、不自行二次发布。

⑥ 复盘模板（半页纸，别写成流水账）

```
## 积分明细查询 v2 上线复盘（2026-11-13）

1. 结果：按计划上线，灰度 100%，P99 78ms，错误率 0，零回滚。
2. 数据：200QPS 压测 P99 78ms / 命中率 71% / CPU 42%。
3. 做对了什么（可复用）：提前定接口契约 → 联调零返工；缓存只缓存前 3 页 → key 量可控。
4. 哪里返工了：周二 count 与 list 用了两个不同时间窗，导致总数与列表对不上（根因：参数各算一次）。
   改进：时间窗在 Exe 入口算一次，向下传值，禁止各层重算。
5. 下周要补的：深分页改游标（预计 1 天）；补 2 个监控指标；把本次缓存模板沉淀成团队公共组件。
```

交付物与验收

- **交付物：**变更单（已闭合）、监控看板链接、复盘文档、治理清单。
- **验收：**生产灰度 100% 且观察 1 小时无异常；告警实测能收到；复盘里至少有一条"下次怎么避免"的具体动作。

04 给你的几句话

排期是给事情的，这几句是给人的。前三个月，方法比产出更值钱。

记住这条	具体是什么意思
先跑通，再优化	周二不要碰缓存，周三不要碰压测。每个阶段只解决一个问题，否则你会在四个问题之间来回跳，一个都没解决。
卡住 30 分钟就喊人	不是能力问题，是成本问题。你卡 3 小时，团队损失 3 小时；问一句，损失 10 分钟。但问之前，把你的尝试过程写清楚。
交付物思维	每天问自己："今天结束时，我能拿出什么给别人看？"拿不出来，说明今天在空转。
不要藏问题	周五晚上暴露的问题，比周三早上暴露的问题，代价差 10 倍。落后了就说落后了，我来调范围。
把踩的坑记下来	每天 18:00 那 30 分钟，写三行：今天踩了什么坑、根因、下次怎么做。三个月后这就是你比别人快的原因。

下周预告（先有个数，不用现在做）： ① 深分页改游标分页；② 把本次缓存实现沉淀成团队公共组件；③ 参与一次交易域的需求评审，只旁听，练"听懂需求背后的真实诉求"。

有问题随时找我。周一 10:30 前把设计文档发我，我们过一遍再动手。
—— 你的 Leader

本文档为 2026 年第 46 周工作安排（2026-11-09 至 2026-11-13）。任务编号对应 AONE-88217 子任务，进度以任务单状态为准。文档中代码为结构示意，包名、框架封装（Tair / Diamond / HSF）以团队现有工程规范为准。生成时间：2026-11-09。